

第 10 讲：MLP / Feed-Forward Neural Network

把神经元、Linear 与 Activation 真正组装成一个网络

- 真正理解 Input -> Linear -> Activation -> Linear -> Output 的每一步。
- 用一个 3 -> 4 -> 2 的小网络逐个数字手算 Forward Pass。
- 掌握隐藏层、宽度、层数、参数量与 Shape 的基本含义。
- 连接 Transformer FFN / MLP：以后看到 FFN 不再把它当成黑盒。
- 实验：NumPy 手写、Batch / 3D Tensor、PyTorch nn.Sequential 对照。

学习方法：先看懂每一步数字和 Shape，再让 NumPy / PyTorch 帮我们自动计算。

1. 【原理】MLP 到底是什么？

MLP 是 Multi-Layer Perceptron（多层感知机）的缩写。对我们当前的学习目标，可以先把它理解为：把多个 Linear 层串起来，并在中间加入 Activation Function。

A minimal MLP / Feed-Forward Network



Linear mixes features; activation introduces nonlinearity.

最低必要心智模型

MLP 不是“会思考的小盒子”。它首先就是一连串确定的数值计算：矩阵乘法 -> 加 Bias -> 激活函数 -> 再做矩阵乘法。参数通过后续训练被学习出来。

你在之前的 LLM 课程里已经见过 Transformer FFN / MLP。现在我们第一次从最底层重新组装它。

2. 【通俗解释】为什么要有“隐藏层”？

假设输入只有 3 个数字，但这 3 个数字并不一定直接适合解决最终任务。第一层 Linear 可以把原来的 3 个数字重新混合成 4 个新的中间特征；ReLU 再根据数值正负进行非线性筛选；第二层 Linear 再把这些中间特征组合成最终 2 个输出。

阶段	Shape	你可以暂时怎样理解
输入 x	(3,)	原始的 3 个特征
第一层 z1	(4,)	把 3 个特征重新组合成 4 个中间量
激活 a1	(4,)	对 4 个中间量逐元素做非线性处理
输出 y	(2,)	再把 4 个隐藏特征组合成 2 个最终输出

注意

“隐藏特征”是数学上的内部表示。不要把单个隐藏神经元直接解释成某个明确的人类概念。真实网络里，一个语义通常分布在大量维度和层之间。

3. 【具体数字例子】先把整个网络写出来

我们构造一个最小 MLP：输入维度 3，隐藏维度 4，输出维度 2。

```
x = [1, 2, -1]
```

```
W1 = [[ 1,    0, -1,  0.5],  
       [ 0.5,  1,  0, -1.0],  
       [ 1, -0.5,  1,  1.0]]  
b1 = [0, 0.5, 1, -0.5]
```

```
W2 = [[ 1, -1],  
       [0.5, 0.5],  
       [-1,  1],  
       [ 2,  0]]  
b2 = [0.2, -0.3]
```

第一层：3 → 4；第二层：4 → 2。也就是说，第一层有 4 个隐藏神经元，第二层有 2 个输出神经元。

Shape tracking for one sample



The hidden width changes 3 → 4, while ReLU keeps the shape unchanged.

4. 【手算】第一层 Linear： $z1 = xW1 + b1$

第一层的每一个输出，本质上都是“输入向量 x 与 $W1$ 的某一列做点积，再加对应 Bias”。

隐藏神经元	点积 + Bias	结果
h0	$1*1 + 2*0.5 + (-1)*1 + 0$	1
h1	$1*0 + 2*1 + (-1)*(-0.5) + 0.5$	3
h2	$1*(-1) + 2*0 + (-1)*1 + 1$	-1
h3	$1*0.5 + 2*(-1) + (-1)*1 - 0.5$	-3

所以：

$$z1 = [1, 3, -1, -3]$$

Shape 从 (3,) 变成 (4,)。第一层改变了“特征维度”。

和第 4/5/7 讲的连接

这里没有出现新的数学规则。你仍然只是在做 Dot Product、Matrix Multiplication、Weight 和 Bias。MLP 的新东西在于：把这些组件按层组合起来。

5. 【手算】ReLU： $a1 = \text{ReLU}(z1)$

第9讲学过 $\text{ReLU}(x) = \max(0, x)$ 。它逐元素处理 $z1$ ，不改变 Shape。

位置	$z1$	ReLU 后 $a1$
$h0$	1	1
$h1$	3	3
$h2$	-1	0
$h3$	-3	0

$z1 = [1, 3, -1, -3]$

$a1 = [1, 3, 0, 0]$

这一步虽然没有改变 Shape，但改变了第二层看到的信息： $h2$ 和 $h3$ 在这个输入上被 ReLU 截断为 0。

重要直觉

Linear 负责“混合和重组特征”；Activation 负责“引入非线性”。两者结合后，网络才能表达复杂的、不能折叠成单一 affine 的函数。

6. 【手算】第二层 Linear： $y = a1W2 + b2$

现在第二层接收到的是 $a1=[1,3,0,0]$ 。两个输出分别是：

$$\begin{aligned} y0 &= 1*1 + 3*0.5 + 0*(-1) + 0*2 + 0.2 \\ &= 2.7 \end{aligned}$$

$$\begin{aligned} y1 &= 1*(-1) + 3*0.5 + 0*1 + 0*0 - 0.3 \\ &= 0.2 \end{aligned}$$

最终：

$$y = [2.7, 0.2]$$

到这里，一个完整的 Forward Pass 已经结束。注意：我们目前只做“前向计算”，还没有讨论 Loss、Gradient 和参数更新。那些内容将在后续阶段加入。

7. 【必要公式】一个两层 MLP 的标准写法

$$\begin{aligned} z1 &= x \cdot W1 + b1 \\ a1 &= \text{phi}(z1) \\ y &= a1 \cdot W2 + b2 \end{aligned}$$

其中 phi 表示激活函数，例如 ReLU、GELU 或 SiLU。

符号	含义	本例 Shape
x	输入向量	(3,)
W1	第一层权重	(3,4)
b1	第一层 Bias	(4,)
z1	第一层激活前输出	(4,)
a1	隐藏层激活	(4,)
W2	第二层权重	(4,2)
b2	第二层 Bias	(2,)
y	最终输出	(2,)

公式阅读法

以后论文里遇到 MLP 公式，不要先问“这是什么高级网络”，而要先做三件事：①写出每个 Tensor 的 Shape；②检查相邻矩阵维度能否相乘；③判断 Activation 放在哪里。

8. 参数量：为什么隐藏维度变大，模型参数会迅速增加？

本例第一层 3 -> 4：Weight 有 $3 \times 4 = 12$ 个数，Bias 有 4 个数，共 16 个参数。第二层 4 -> 2：Weight 有 $4 \times 2 = 8$ 个数，Bias 有 2 个数，共 10 个参数。

$$\blacksquare = (3 \times 4 + 4) + (4 \times 2 + 2) = 26$$

一般地，两层 MLP 的参数量约为：

$$\begin{aligned} &D_{in} * D_{hidden} + D_{hidden} \\ &+ D_{hidden} * D_{out} + D_{out} \end{aligned}$$

当 D_{hidden} 很大时，两侧的大矩阵会占主要参数。这一点以后理解 Transformer FFN 为什么参数很多非常重要。

LLM 连接

许多 Transformer 中，FFN 会先把 hidden size 扩大，再投影回来。例如概念上可以是 $D \rightarrow 4D \rightarrow D$ 。即使 Attention 很显眼，FFN 也可能占据模型参数和计算中的很大一部分。

9. Batch 与 LLM 三维 Tensor：MLP 怎样处理很多 Token？

第 6 讲已经学过：Linear 作用在最后一个维度。MLP 只是把两个 Linear 和一个逐元素 Activation 串起来，因此同样可以自然处理 Batch 和三维 Tensor。

■■■■■	(3,)	->	(4,)	->	(4,)	->	(2,)
Batch■	(B, 3)	->	(B, 4)	->	(B, 4)	->	(B, 2)
LLM■	(B, T, 3)	->	(B, T, 4)	->	(B, T, 4)	->	(B, T, 2)

B 表示 batch size, T 表示 sequence length。普通 MLP/FFN 对每一个 Token 的最后一维使用同一套参数。

关键区别

普通 token-wise MLP 自己不会让不同 Token 相互交换信息。Transformer 中不同 Token 的交互主要由 Attention 完成，而 FFN/MLP 更像是对每个 Token 的特征表示进行非线性加工。

10. Width 与 Depth：网络为什么叫“多层”？

Width（宽度）通常指某一隐藏层拥有多少个隐藏单元/特征维度；Depth（深度）通常指网络叠了多少层。不同论文的计数习惯可能略有差异，因此要看作者定义。

概念	本例	直觉
输入维度	3	每个样本一开始有 3 个数字
隐藏宽度	4	网络内部暂时使用 4 个中间特征
输出维度	2	最后输出 2 个数字
深度	至少两个 Linear 层	信息经过多次参数化变换

更宽不自动等于更好，更深也不自动等于更好。它们会同时影响表达能力、参数量、优化难度、计算量和过拟合风险。后续学习 Gradient、Normalization、Regularization 时会重新遇到这些问题。

11. 与 Transformer / LLM 的关系：FFN 就是更大的 MLP

你以前在 LLM 课程里已经见过 Transformer Block 中的 FFN / MLP。现在可以把它还原成我们刚刚学习的组件。

```
Transformer hidden state X
      |
      v
Linear: D -> D_ff
      |
      GELU / SiLU / gated activation
      |
      v
Linear: D_ff -> D
      |
      v
new hidden state
```

真实 LLM 可能使用 GELU、SiLU、SwiGLU 等不同形式，Bias 也可能被省略，具体宽度比例也不同。但底层仍然是“线性投影 + 非线性 + 线性投影”这一族结构。

以后读论文时看什么？

当论文说“我们修改 FFN/MLP”，先检查：hidden dimension 是否变化？Activation 是否变化？是否使用 gating？参数量是否变化？计算量是否变化？Ablation 是否控制了这些混杂因素？

12. 【代码实验】从 NumPy 到 PyTorch

在 NumPy 中，我们显式写出每一步：

```
z1 = x @ W1 + b1
a1 = np.maximum(z1, 0)
y = a1 @ W2 + b2
```

在 PyTorch 中，可以把同一结构封装成：

```
model = nn.Sequential(
    nn.Linear(3, 4),
    nn.ReLU(),
    nn.Linear(4, 2),
)

y = model(x)
```

PyTorch 并没有改变数学结构，只是帮我们封装参数存储、批量计算，并在以后通过 Autograd 自动记录计算图。

本讲实验会把 NumPy 权重复制到 PyTorch 中，验证二者输出一致。

13. 【常见误区】

误区	正确理解
“ 隐藏层里每个神经元都有明确语义 ”	不能简单这样解释。内部表示通常是分布式的。
“ 层数越多越聪明 ”	深度只是结构超参数之一，还要考虑数据、优化、归一化、参数规模等。
“ Activation 会改变维度 ”	常见 ReLU/GELU/SiLU 是逐元素函数，通常保持 Shape。
“ MLP 会让 Token 之间通信 ”	普通 token-wise MLP 主要处理每个 Token 的最后一维；跨 Token 交互由 Attention 等机制完成。
“ 参数越多一定效果越好 ”	更大容量也可能增加计算成本和过拟合风险，效果取决于完整训练条件。

科研阅读提醒

论文报告某个 MLP 变体更好时，要区分 “ 结构本身更好 ” 与 “ 只是参数更多 / 计算更多 / 训练设置不同 ”。Ablation 的一个核心目的，就是尽量拆开这些因素。

14. 【实验】本讲要验证什么？

实验不是为了“跑一个神经网络就算完成”，而是围绕可验证问题组织：

实验问题	验证方式
MLP 的每一步数值是否能手算复现？	手算 z_1 、 a_1 、 y ，并与 NumPy 对照
隐藏层每个神经元到底怎样得到输出？	逐列计算 W_1 的贡献
参数量公式是否正确？	手算 26，再与 PyTorch numel() 对照
Batch / 3D Tensor Shape 如何变化？	运行 (B,D) 与 (B,T,D) 输入
PyTorch 是否只是封装了同一数学？	复制同样参数，对比 NumPy / PyTorch 输出

完整实验设计和代码已放在材料包中。建议先在纸上预测输出 Shape，再运行代码。

15. 【本讲总结】从“层”回到具体数字

如果这一讲真正掌握，你应该能把“MLP”这个名词拆成下面的数值链：

```
x
-> x @ W1 + b1
-> activation(...)
-> hidden representation
-> hidden @ W2 + b2
-> y
```

你应该能够回答：每个 Weight 的 Shape 为什么这样设计？隐藏维度为什么变化？Activation 为什么不改变 Shape？参数量怎样算？三维 LLM Tensor 为什么也能直接通过 MLP？

下一讲预告

第 11 讲将正式学习 Forward Propagation 与 Computational Graph：不仅知道“模型算出了 y”，还要把整个计算过程画成图，理解中间节点、数据依赖和以后 Backpropagation 为什么能够沿图反向计算。

最低验收：不看代码，你能手算本讲的 3 -> 4 -> 2 MLP；看到任意两层 MLP，你能先写出各 Tensor 的 Shape 和参数量。